Arquitetura de Integração para Empresa de Serviços

Projeto Aplicado ArchCorp · Cenário 4 · Locação de veículos

Roger Quinelato
Raquel Pereira
Versão de 29 de outubro de 2026
Entrega pronta prevista para 09/11/2026
Apresentação prevista para 23/11/2026

Resumo

O Cenário 4 do Projeto Aplicado descreve uma empresa de serviços com CRM, sistema de contratos, sistema financeiro, sistema de atendimento e sistema de gestão de processos que não se comunicam adequadamente (ARCHCORP, 2026a). O trabalho aplica o cenário a uma locadora de veículos fictícia inspirada na Localiza, com dados sintéticos e sem acesso a sistemas reais. O problema consiste em duplicidade de cadastro, lançamentos manuais, falhas difíceis de rastrear e processos fragmentados.

A solução proposta é uma arquitetura orientada a serviços entregue como monólito modular, com arquitetura hexagonal, APIs REST para interações imediatas e eventos para efeitos desacoplados. A metodologia combinou análise do enunciado, levantamento de requisitos, modelagem AS-IS e TO-BE, registro de decisões em ADR, implementação incremental e verificação automatizada. O protótipo separa os contextos CRM, Contracts, Finance, Support, Workflow e Integration, e a entrega confiável usa outbox, inbox, idempotência, correlação e reprocessamento auditado.

A análise considera a versão 534eab2. Em 29/10/2026, a verificação consolidada registrou 269 aprovados e 14 ignorados (SQLite), 19 testes da interface aprovados, contrato OpenAPI sincronizado e os roteiros F1, F2 e F3 concluídos sem falha; a CI do pull request executa os jobs de aplicação, PostgreSQL 16, RabbitMQ e navegador. A medição do RNF-07, com 60 clientes e 100 amostras em 10 consultas de negócio, apresentou maior p95 de 21,53 ms, abaixo da meta de 500 ms. Um relatório operacional converte métricas, auditoria e eventos em indicadores com decisão sugerida. Os resultados usam dados sintéticos e uma demonstração local (PROJETO APLICADO 7, 2026a, 2026c).

Palavras-chave: arquitetura de sistemas corporativos; SOA; monólito modular; integração de sistemas; interoperabilidade; eventos.

1 Introdução

1.1 Contextualização

Organizações que crescem rapidamente costumam adotar uma aplicação por área. Vendas, contratos, financeiro, atendimento e operações passam a guardar versões próprias dos mesmos fatos, e a integração acontece por recadastro, planilhas ou chamadas ponto a ponto. O enunciado do projeto descreve esse quadro e pede uma proposta arquitetural que mostre como aplicações, serviços e componentes podem trabalhar de forma integrada (ARCHCORP, 2026a).

O problema arquitetural identificado é de propriedade de dados e de coordenação entre sistemas. Uma chamada bem-sucedida não basta quando a ativação de uma locação precisa gerar cobrança e preparação de retirada: uma falha intermediária pode perder o fato ou repetir o efeito.

1.2 Justificativa

Sem fronteiras e contratos definidos, cada nova integração aumenta o acoplamento e o custo de manutenção. O próprio enunciado delimita o objetivo desta etapa: "Nesta primeira entrega, a prioridade não é construir um sistema corporativo completo" (ARCHCORP, 2026a, p. 1). A modernização precisa, portanto, demonstrar decisões arquiteturais verificáveis, e não substituir todos os sistemas.

1.3 Objetivos

O objetivo geral é propor e demonstrar uma arquitetura executável que integre os cinco sistemas do Cenário 4 preservando a fonte oficial de cada dado. Os objetivos específicos são:

- documentar o cenário atual (AS-IS) e os requisitos funcionais e não funcionais priorizados;
- projetar a arquitetura proposta (TO-BE) e justificar o estilo arquitetural escolhido;
- definir e implementar três integrações com contratos versionados e tratamento de erros;
- avaliar nove atributos de qualidade e definir estratégias de escalabilidade, manutenção e evolução;
- relacionar a proposta ao valor de negócio e à viabilidade para a organização.

1.4 Apresentação do relatório

A seção 2 apresenta a fundamentação teórica; a seção 3, a metodologia; a seção 4 descreve e analisa os resultados (cenário atual, requisitos, arquitetura proposta, sistemas corporativos, interoperabilidade, integrações, qualidade e evolução); a seção 5 traz as considerações finais. O repositório contém OpenAPI, AsyncAPI, ADRs, testes e roteiros de demonstração.

2 Fundamentação teórica

2.1 Arquitetura de sistemas

A arquitetura de software reúne as estruturas de um sistema, seus elementos, as relações entre eles e as propriedades desses elementos e relações. Bass, Clements e Kazman (2021) tratam as decisões arquiteturais como meios de atender a requisitos de qualidade e de controlar riscos, avaliados por cenários com estímulo, ambiente, resposta e medida. Por isso, este trabalho liga cada escolha a uma necessidade do cenário e a uma evidência verificável.

2.2 Sistemas corporativos: ERP, CRM e BPM

Sistemas corporativos apoiam processos que atravessam a organização. O ERP (Enterprise Resource Planning) integra em uma base comum dados financeiros, de compras, de pessoal e de operações. Davenport (1998) mostra que esse ganho de integração tem custo: o pacote impõe sua própria lógica aos processos e exige que a organização alinhe estratégia e sistema. No cenário estudado, financeiro e faturamento cumprem o papel típico de um módulo de ERP.

O CRM (Customer Relationship Management) concentra o relacionamento com o cliente. Buttle e Maklan (2019) distinguem o CRM estratégico, o operacional, que automatiza vendas, marketing e serviço, e o analítico, que explora os dados de clientes. Aqui o CRM é a fonte oficial de cliente, contatos, consentimentos e oportunidades.

O BPM (Business Process Management) trata o processo de ponta a ponta como objeto de gestão. Dumas et al. (2018) descrevem um ciclo de identificação, descoberta, análise, redesenho, implementação e monitoramento de processos. O contexto Workflow do protótipo implementa uma versão mínima dessa ideia: instâncias, tarefas, responsáveis e prazos iniciados por eventos de negócio, sem motor BPM externo.

2.3 Arquitetura em camadas

A arquitetura em camadas separa apresentação, aplicação, domínio e acesso a dados, e cada camada depende apenas da inferior (FOWLER, 2002). Richards e Ford (2020) destacam a simplicidade e o baixo custo do estilo, mas também o risco de acoplamento técnico: uma mudança de negócio atravessa todas as camadas. O protótipo mantém camadas dentro de cada contexto e usa os contextos de negócio como divisão principal.

2.4 SOA e serviços

A arquitetura orientada a serviços (SOA) expõe capacidades de negócio por contratos estáveis, de modo que consumidores não dependam da implementação interna. Neste projeto, serviço significa uma fronteira de negócio com interface pública e propriedade de dados. A implantação inicial pode reunir essas fronteiras em um único processo sem eliminar a separação lógica.

2.5 Microsserviços e arquiteturas distribuídas

Lewis e Fowler (2014, on-line) definem o estilo de microsserviços como "an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API". A distribuição permite implantar e escalar cada serviço separadamente, mas acrescenta latência de rede, falhas parciais, consistência eventual e custo operacional (NEWMAN, 2021). Fowler (2015, on-line) observa que "almost all the successful microservice stories have started with a monolith that got too big and was broken up".

O enunciado faz a mesma advertência: "A equipe não deverá utilizar microsserviços apenas por serem uma tecnologia atual" (ARCHCORP, 2026a, p. 5). Essas referências sustentam a escolha de um monólito modular com fronteiras extraíveis, descrita na seção 4.3.

2.6 Arquitetura hexagonal e monólito modular

Na arquitetura hexagonal, a aplicação define portas, e adaptadores conectam essas portas a HTTP, bancos, filas e sistemas externos (COCKBURN, 2005). As regras de domínio ficam independentes da infraestrutura, e simuladores podem substituir conectores reais em testes. O monólito modular aplica a mesma disciplina de fronteiras em um único implantável: reduz a operação inicial e preserva limites que podem orientar uma extração futura.

2.7 APIs, HTTP e eventos

A RFC 9110 descreve o HTTP como "a stateless application-level protocol for distributed, collaborative, hypertext information systems" (FIELDING; NOTTINGHAM; RESCHKE, 2022, on-line). O projeto usa REST/JSON quando o solicitante precisa do resultado imediato. Eventos representam fatos já confirmados e permitem vários consumidores sem que o produtor os conheça. A especificação OpenAPI descreve as rotas HTTP, e a AsyncAPI descreve canais, mensagens e payloads (OPENAPI INITIATIVE, 2024; ASYNCAPI INITIATIVE, 2023).

2.8 Integração de sistemas

Hohpe e Woolf (2003) organizam a integração por mensagens em canais, mensagens, roteamento, transformação e pontos de extremidade. Dois padrões derivados sustentam a confiabilidade do protótipo. A outbox transacional grava o evento na mesma transação da mudança de negócio. A inbox registra o evento recebido por consumidor. A combinação oferece entrega ao menos uma vez sem repetir o efeito de negócio. Nygard (2018) acrescenta padrões de estabilidade, como timeout, retentativa limitada e isolamento de falhas, aplicados à leitura do CRM e à consulta de contratos.

2.9 Interoperabilidade

Interoperabilidade exige mais que conectividade técnica. Os sistemas precisam concordar sobre identidade das entidades, significado dos campos, formato, versão, tratamento de erro e fonte oficial. O modelo canônico deve conter apenas os campos compartilhados por cada fluxo, para não se tornar um modelo central rígido que todos os sistemas precisem acompanhar.

2.10 Qualidade, escalabilidade, manutenibilidade e evolução

A ISO/IEC 25010 organiza a qualidade de produto em características como eficiência de desempenho, compatibilidade (que inclui interoperabilidade), confiabilidade, segurança e manutenibilidade (ISO/IEC, 2011). Escalabilidade é a capacidade de atender mais carga acrescentando recursos ao componente pressionado. Manutenibilidade é a facilidade de modificar o sistema com impacto localizado. Quanto à evolução, Lehman (1980) formulou que sistemas em uso precisam mudar continuamente e que sua complexidade cresce se nada for feito para contê-la. Essa observação justifica contratos versionados, migrações aditivas e ADRs como mecanismos permanentes, e não como tarefas de fim de projeto.

3 Metodologia

3.1 Participantes

Integrante	Contribuição
Roger Quinelato	Backend, produto e negócio
Raquel Pereira	Frontend e levantamento de requisitos

3.2 Recursos utilizados

Finalidade	Recurso
Versionamento	Git e GitHub (issues, pull requests e Actions)
Gestão do trabalho	Jira (registro principal), Trello, Notion e Google Drive
Desenvolvimento	Python, FastAPI, SQLAlchemy, Alembic; React e TypeScript na interface
Persistência e mensageria	PostgreSQL e RabbitMQ no Docker Compose; SQLite na demonstração local e nos testes
Contratos	Swagger/OpenAPI 3.1 e AsyncAPI 3.0
Modelagem e documentação	Mermaid, Markdown, python-docx e LibreOffice
Verificação	Pytest, testes de contrato e de fronteira, Playwright com axe, roteiros httpx e agentes de revisão FDE e QA somente leitura

3.3 Métodos

Atividade	Método
Levantamento de requisitos	Leitura do enunciado, extração de RFs e RNFs e critérios de aceite mensuráveis
Análise do cenário	Mapeamento de sistemas, usuários, dados e dependências; premissas marcadas como tal
Modelagem da arquitetura	Diagramas AS-IS e TO-BE, DDD estratégico e ADRs com alternativas
Definição dos componentes	Contextos por propriedade de dados, com portas públicas
Definição das integrações	Fluxos F1 a F3 com origem, destino, dados, comunicação e erros
Desenvolvimento do protótipo	Entregas incrementais por tarefa, revisão por pull request e CI
Avaliação da arquitetura	Cenários de qualidade, testes automatizados, roteiros e medição local

3.4 Procedimentos metodológicos

Os procedimentos compreenderam análise do escopo, levantamento de requisitos e AS-IS, modelagem TO-BE e contratos, implementação dos contextos e avaliação dos fluxos e atributos de qualidade. A matriz de rastreabilidade relaciona requisito, decisão, implementação e evidência. O enunciado prevalece sobre premissas, e detalhes organizacionais ausentes permanecem como hipóteses a validar.

O recorte documental é 29 de outubro de 2026, com referência técnica na versão 534eab2. Cada mudança passou por um portão de qualidade determinístico (testes, contrato e CI) e por revisão independente: um agente de QA verifica o código e um agente FDE confronta a entrega com o Guia, ambos sem permissão de alterar arquivos. O planejamento estabelece a entrega pronta em 09/11/2026 e a apresentação em 23/11/2026. Esses marcos são compromissos previstos, sem declaração de submissão, apresentação realizada ou aceite docente. A conclusão formal depende também da identificação dos participantes e de suas contribuições.

4 Descrição e análise dos resultados

4.1 Cenário atual (AS-IS)

O enunciado confirma os cinco sistemas e os problemas gerais, mas não informa produtos, interfaces ou tecnologias (ARCHCORP, 2026a). A Figura 1 mostra a hipótese de trabalho: as áreas se comunicam por recadastro, planilhas e acionamentos manuais. Essa representação é premissa de modelagem, não observação de campo.

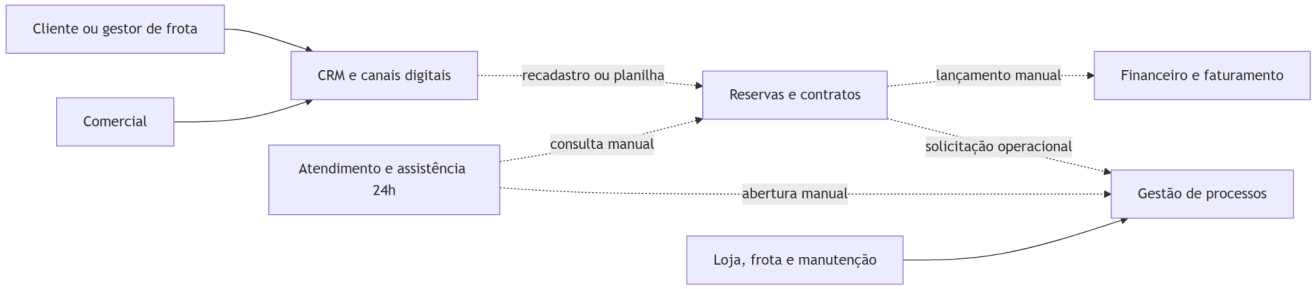


Figura 1 — Arquitetura atual (AS-IS) e trocas manuais entre áreas. Fonte: elaborada pela equipe.

Sistema	Usuários	Dados produzidos ou consumidos
CRM	Comercial e canais digitais	Cliente, contatos, consentimentos e oportunidade
Reservas e contratos	Reservas e área de contratos	Reserva, contrato, grupo de veículo, vigência, proteções e SLA
Financeiro e faturamento	Financeiro e cobrança	Cobrança, fatura, pagamento, caução e inadimplência
Atendimento e assistência 24h	Atendentes	Chamado, assistência, prioridade, histórico e resolução
Gestão de processos	Operações, loja e frota	Tarefa, responsável, prazo e estado do processo

4.1.1 Dependências e problemas

Etapa	Origem	Destino	Comunicação atual (premissa)	Problema
Venda	CRM	Reservas e contratos	Recadastro ou planilha	Duplicidade e ID divergente
Ativação	Reservas e contratos	Financeiro	Lançamento manual	Cobrança tardia ou ausente
Retirada	Reservas e contratos	Gestão de processos	Solicitação manual	Início sem rastreabilidade
Suporte	Atendimento	Reservas e contratos	Consulta manual	SLA incorreto
Resolução	Atendimento	Gestão de processos	Abertura manual	Processo fragmentado

4.1.2 Limitações da arquitetura atual e necessidades de integração

- Não há identidade comum: o mesmo cliente recebe códigos diferentes em cada sistema.
- Não há contratos de integração: cada troca depende de pessoas e formatos informais.
- Falhas não são rastreáveis: não existe correlação entre a venda, a cobrança e o atendimento.
- Mudanças em um sistema afetam os demais sem aviso, o que dificulta manutenção e evolução.

Daí derivam as necessidades: identidade global com vínculo aos identificadores legados, contratos versionados, entrega confiável com repetição segura, rastreabilidade por correlação e fronteiras que preservem a fonte oficial. A arquitetura admite adaptadores diferentes quando a descoberta real identificar APIs, arquivos, webhooks ou bancos legados.

4.2 Requisitos

4.2.1 Requisitos funcionais

ID	Requisito	Critério de aceite
RF-01	Referência única de cliente	UUID global e IDs legados associados
RF-02	Reserva e contrato a partir do CRM	Rascunho sem recadastro, pelo customerId do CRM
RF-03	Ativar, faturar e preparar retirada	Uma fatura e um processo mesmo com repetição
RF-04	Chamado com contrato e SLA	Elegibilidade e prazo registrados; pendente se Contracts cair
RF-05	Propagar alteração cadastral	CustomerUpdated.v1 consumido e auditado
RF-06	Consultar operação	Trilha completa por correlationId
RF-07	Reprocessar falha	Motivo consultável e reprocessamento sem duplicar efeito

4.2.2 Requisitos não funcionais e prioridade

ID	Atributo	Prioridade	Por que é prioritário no cenário
RNF-01	Interoperabilidade	P0	Cinco sistemas com modelos e identificadores diferentes
RNF-02	Confiabilidade e tolerância a falhas	P0	Cobrança duplicada ou perdida tem impacto financeiro direto
RNF-03	Segurança e privacidade	P0	Dados pessoais de clientes e papéis distintos por área
RNF-04	Observabilidade	P0	Operações atravessam módulos e precisam de diagnóstico
RNF-05	Manutenibilidade	P0	Equipe júnior e evolução contínua do protótipo
RNF-06	Disponibilidade	P0	Assistência 24h não pode perder chamado
RNF-07	Desempenho	P0	p95 local abaixo de 500 ms na demonstração
RNF-08	Escalabilidade	P1	Depende de volumes reais ainda não informados
RNF-09	Testabilidade	P0	Fluxos e falhas precisam ser reproduzíveis antes da apresentação

P0 indica requisito obrigatório nesta entrega; P1 indica requisito preservado no desenho, mas dependente de métricas reais. Interoperabilidade e confiabilidade são os mais críticos, porque o problema central do cenário é a integração e porque os efeitos financeiros não podem se repetir nem se perder.

4.3 Arquitetura proposta (TO-BE)

A decisão registrada no ADR-001 adota SOA pragmática entregue como monólito modular com arquitetura hexagonal. CRM, Contracts, Finance, Support e Workflow mantêm regras e dados próprios. Integration cuida de identidade legada, correlação, idempotência, outbox, inbox, auditoria e falhas, sem decidir regras de negócio. A Figura 2 mostra os componentes e os mecanismos de comunicação.

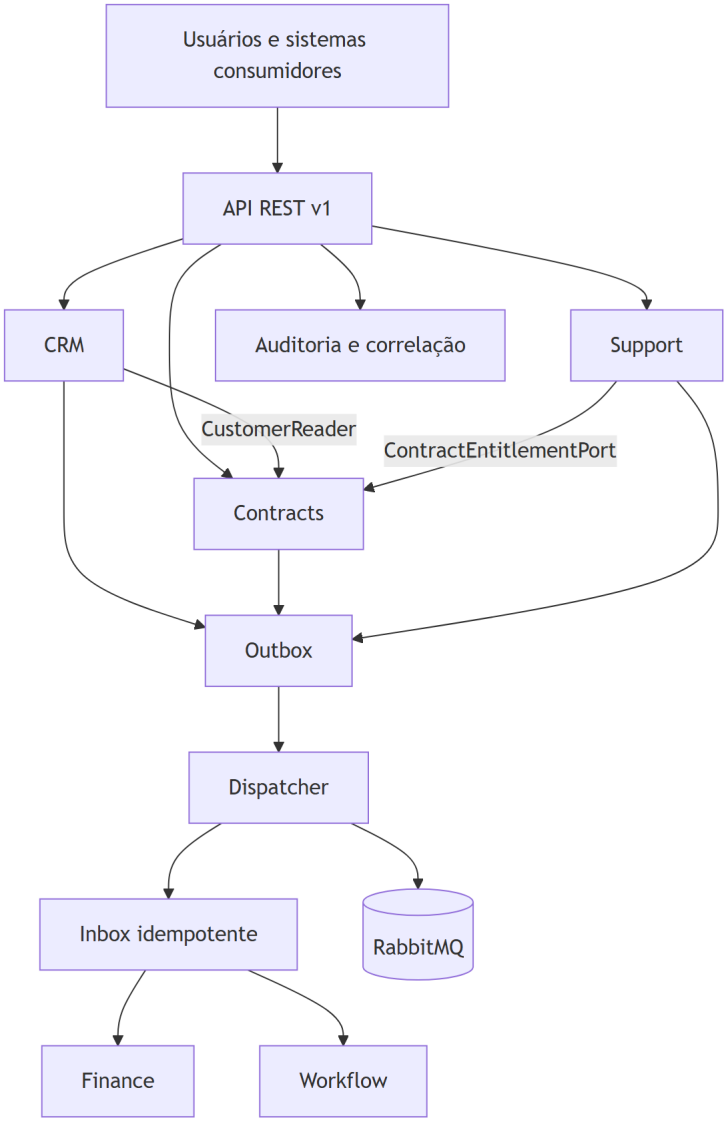


Figura 2 — Diagrama de componentes da arquitetura proposta. Fonte: elaborada pela equipe.

Contexto	Responsabilidade	Interface pública
CRM	Cadastro, consentimento e elegibilidade	CustomerReader e CustomerUpdated.v1
Contracts	Reserva, contrato e SLA	ContractEntitlementPort, ContractActivated.v1 e ContractClosed.v1
Finance	Faturas, pagamentos e inadimplência	Consumidores de ContractActivated e ContractClosed
Support	Chamados, SLA e reconciliação	API de atendimento e TicketOpened.v1
Workflow	Processos, tarefas e prazos	Consumidores de eventos
Integration	Entrega confiável e rastreabilidade	API operacional e fachada integration.public

4.3.1 Implantação e camadas

A Figura 3 mostra a implantação do protótipo. Docker Compose inicia a API, o PostgreSQL e o RabbitMQ; a interface React navega pelos cinco contextos e exibe os erros no formato application/problem+json (RFC 9457) com code e correlationId.

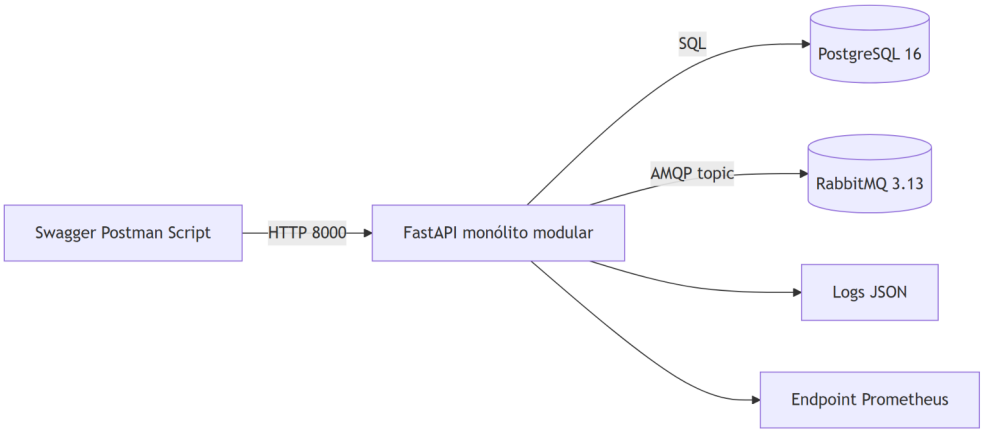


Figura 3 — Implantação do protótipo. Fonte: elaborada pela equipe.

Camada	Elementos	Regra de dependência
Entrada	Rotas REST, autenticação e validação	Converte dados e chama casos de uso
Aplicação	Serviços de cada contexto	Depende de portas públicas
Domínio	Estados e regras	Não depende de HTTP, banco ou broker
Saída	SQLAlchemy, RabbitMQ, adaptadores e logs	Implementa portas e é substituível

4.3.2 Justificativa, vantagens e limitações

O estilo é adequado porque o problema é de integração e propriedade de dados, não de escala extrema: os volumes reais não foram informados e a equipe é júnior. As vantagens são uma única unidade de implantação, transações locais, testes rápidos e fronteiras que permitem extrair um contexto quando métricas justificarem. As limitações são escala conjunta de todos os módulos, banco compartilhado (protegido por convenção, prefixos e teste de fronteira) e uma falha de processo que afeta todos os contextos. O ADR-001 registra as alternativas descartadas: REST ponto a ponto, microsserviços completos e ESB com regras centrais.

4.3.3 Segurança

O protótipo aplica papéis por tokens locais e permite configurar um segredo compartilhado para o modo público. A validação ocorre na fronteira, e eventos não carregam documentos, dados bancários nem a descrição do chamado. A versão de referência compara credenciais em bytes para tratar caracteres não ASCII e limita o registro de exceções inesperadas ao tipo do erro, sem mensagem sensível nem traceback (PROJETO APLICADO 7, 2026b). Um ambiente real exigiria provedor OIDC, contas individuais e TLS verificado; esses itens não estão demonstrados. A demonstração é local, sem publicação pública (ADR-007).

4.4 Sistemas corporativos e interoperabilidade

4.4.1 Papel de cada sistema

Sistema	Categoria	Processos atendidos	Integrações	Papel na arquitetura
CRM	CRM operacional	Cadastro, consentimento, oportunidade	Fornece cliente a Contracts; publica CustomerUpdated	Fonte oficial do cliente
Reservas e contratos	Sistema de contratos	Reserva, contrato, ativação, encerramento	Lê o CRM; publica eventos; responde elegibilidade	Fonte oficial de contrato e SLA
Financeiro e faturamento	Módulo financeiro de ERP	Fatura, pagamento, inadimplência	Consome ativação e encerramento	Fonte oficial de cobrança
Atendimento e assistência 24h	Sistema de atendimento	Chamado, SLA, atribuição, resolução	Consulta Contracts; publica TicketOpened	Fonte oficial de chamado
Gestão de processos	BPM/workflow	Retirada, resolução, tarefas	Consome eventos de contrato e chamado	Fonte oficial do processo

4.4.2 Mecanismos de interoperabilidade

- Padronização de dados: JSON UTF-8, datas ISO 8601 em UTC e valores com moeda explícita.
- Contratos de integração: OpenAPI para REST e AsyncAPI para eventos, com envelope eventId, eventType, eventVersion, occurredAt, correlationId, causationId, producer e payload.
- Identificação de entidades: UUID global por entidade e tabela de identificadores legados por sistema de origem.
- Sincronização: eventos de fato confirmado e projeções locais atualizadas por consumidores idempotentes.
- Tratamento de inconsistências: a fonte oficial prevalece; divergências ficam auditadas, sem regra de "última escrita vence".

4.4.3 Exemplo de problema de interoperabilidade

Problema: o CRM legado identifica o cliente como WEB-LOCALIZA-1001, enquanto o sistema de contratos usa outro código para a mesma pessoa. Ao criar o contrato, o atendente recadastra o cliente e surgem dois registros divergentes.

Solução: o CRM cria o cliente com UUID global e registra o vínculo (CRM, WEB-LOCALIZA-1001) em Integration. Qualquer sistema resolve o código legado por GET /api/v1/integration/legacy-ids/CRM/WEB-LOCALIZA-1001 e recebe o mesmo UUID. Contracts recebe apenas esse UUID e lê os dados pela porta CustomerReader; um segundo cadastro com o mesmo código legado é recusado com 409 CONFLICT. O roteiro do F1 verifica que o ID legado resolve o mesmo customerId do contrato.

4.5 Integração F1: cliente do CRM para contrato

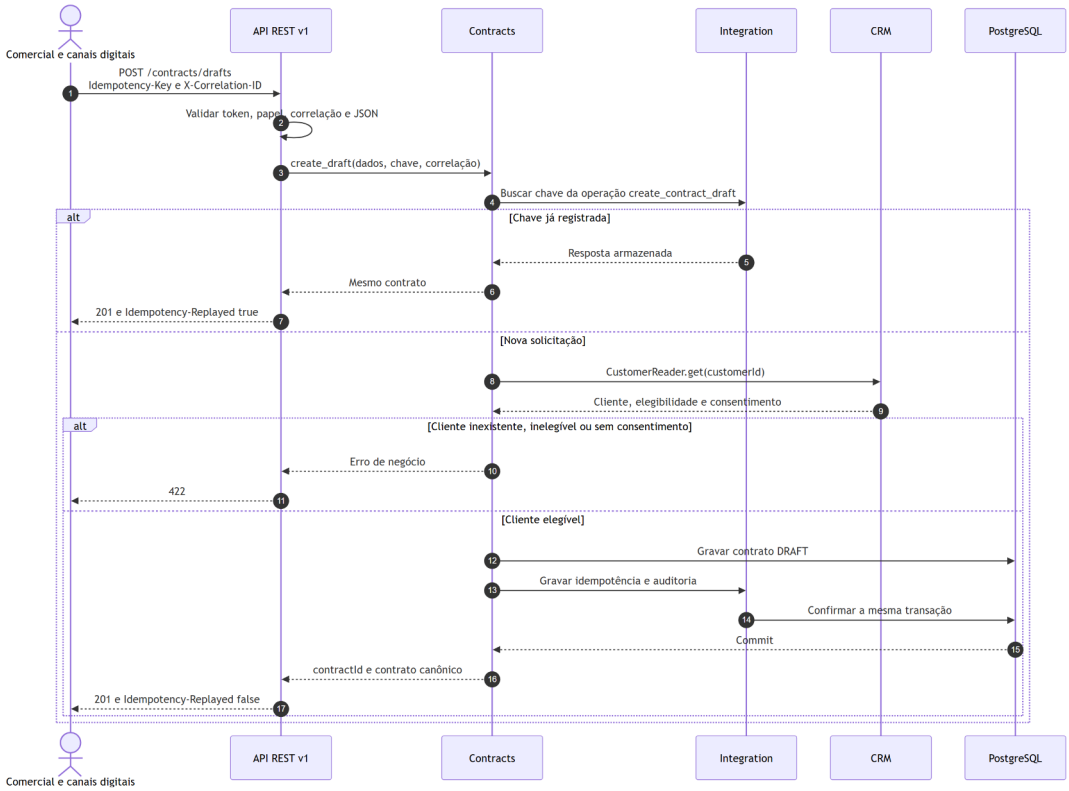


Figura 4 — Sequência do F1. Fonte: elaborada pela equipe.

Item	Definição
Origem e destino	CRM (via canal comercial) → Contracts
Objetivo	Criar reserva e rascunho de contrato sem recadastro
Comunicação	Síncrona: REST/JSON e porta CustomerReader com retentativa limitada
API	POST /contracts/reservations, POST /contracts/reservations/{id}/draft e POST /contracts/drafts
Dados enviados	customerId, serviceCode, período, cobrança, slaHours e Idempotency-Key
Dados recebidos	contractId, status DRAFT, dados canônicos e Idempotency-Replayed
Erros	422 cliente inelegível ou inexistente; 409 chave de idempotência reutilizada com outro pedido

4.6 Integração F2: ativação, cobrança e preparação de retirada

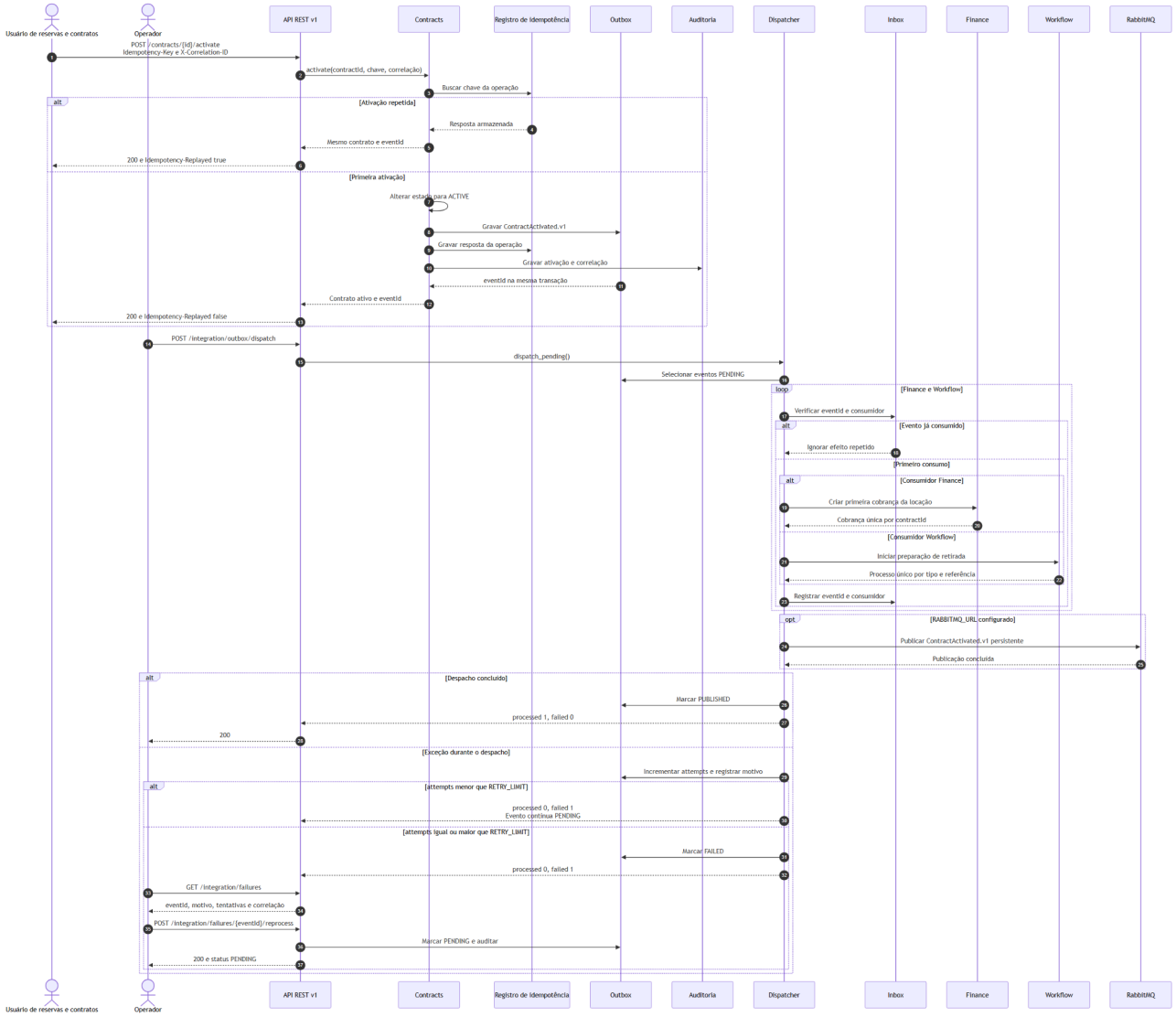


Figura 5 — Sequência do F2 com reentrega e falha permanente. Fonte: elaborada pela equipe.

Item	Definição
Origem e destino	Contracts → Finance e Workflow
Objetivo	Faturar e iniciar a preparação de retirada uma única vez
Comunicação	Assíncrona: ContractActivated.v1 pela outbox; cópia opcional no RabbitMQ
Dados	Envelope do evento com contractId, customerId, cobrança e SLA
Erros	Três tentativas; depois FAILED com motivo genérico; reprocessamento auditado; inbox impede duplicidade

4.7 Integração F3: chamado com contrato e SLA

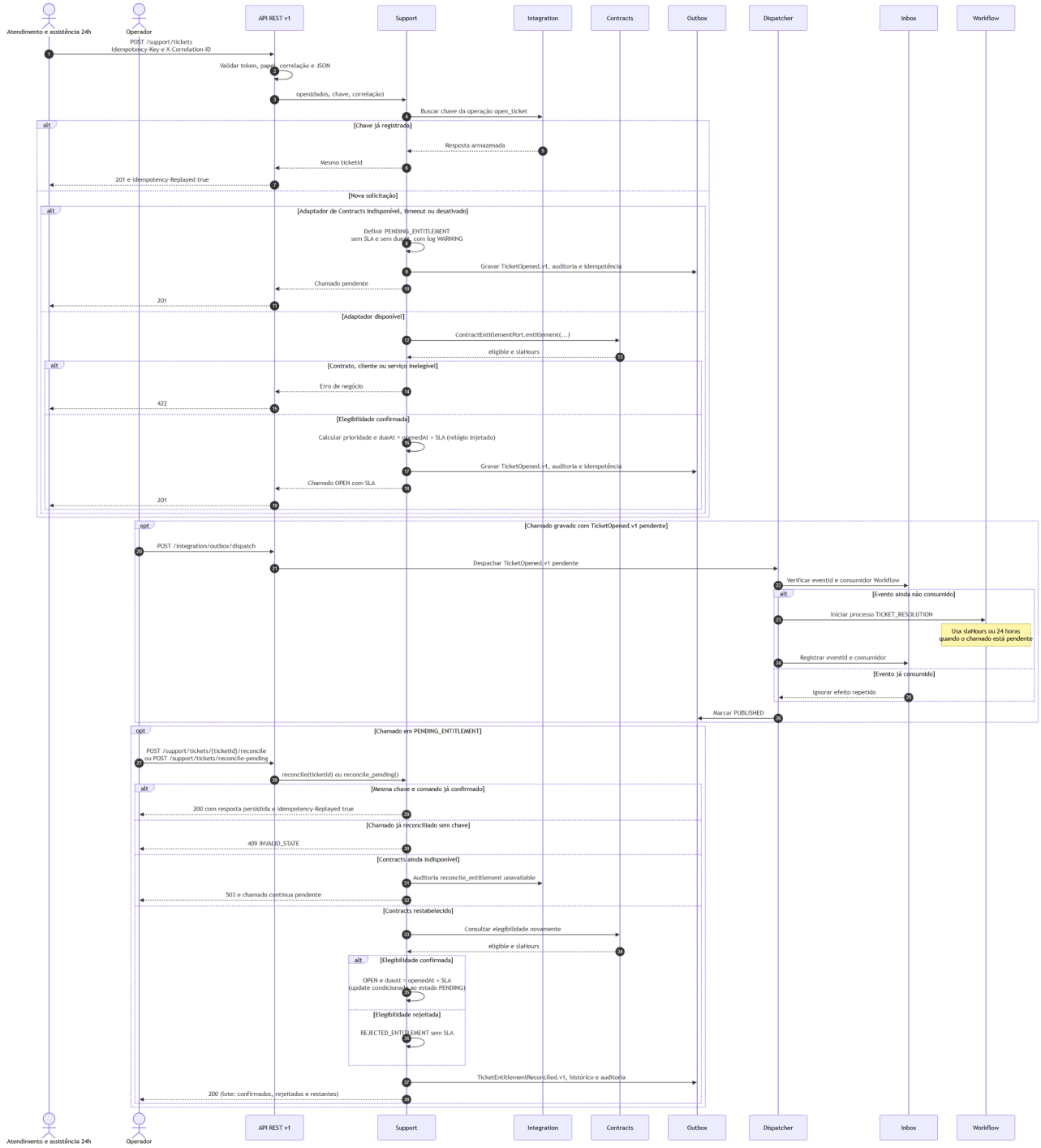


Figura 6 — Sequência do F3 com indisponibilidade e reconciliação. Fonte: elaborada pela equipe.

Item	Definição
Origem e destino	Support → Contracts (síncrono) e Support → Workflow (evento)
Objetivo	Abrir chamado com SLA do contrato e iniciar o processo de resolução
Dados	customerId, contractId, serviceCode, categoria; resposta com prioridade, dueAt e slaStatus
Erros	422 contrato inválido; Contracts indisponível gera PENDING_ENTITLEMENT e reconciliação; 503 enquanto não volta

4.8 Qualidade arquitetural

Atributo	Necessidade	Risco	Estratégia adotada	Resultado esperado
Interoperabilidade	Integrar modelos e IDs diferentes	Vincular cliente errado	UUID global, IDs legados, OpenAPI e AsyncAPI	Troca só dos dados necessários
Confiabilidade	Preservar fatos sem duplicar	Cobrança perdida ou dupla	Outbox, inbox, Idempotency-Key e restrições únicas	Um efeito por fato
Segurança	Restringir operações	Acesso indevido ou dado sensível em log	RBAC, validação e minimização	Só papéis autorizados agem
Observabilidade	Seguir operação entre módulos	Falha sem causa identificável	correlationId, logs, métricas e auditoria	Trilha completa por consulta
Manutenibilidade	Mudar um contexto isoladamente	Dependências cruzadas	Portas públicas e teste de fronteira	Mudanças localizadas
Disponibilidade	Não perder chamado	Contracts indisponível	Estado pendente e reconciliação	Chamado preservado
Desempenho	Resposta rápida na demo	Latência de banco e serialização	API sem estado e índices	p95 máximo de 21,53 ms
Escalabilidade	Crescer sem redesenho	Saturação da API ou da fila	API sem estado e consumidores idempotentes	Escala no componente pressionado
Testabilidade	Reproduzir fluxos e falhas	Erro só na apresentação	Adaptadores substituíveis e suíte automatizada	269 aprovados e 14 ignorados (SQLite)

A medição de desempenho de 29/10/2026 usa SQLite local com dados sintéticos; a CI verifica a mesma base com PostgreSQL 16, RabbitMQ e navegador. A capacidade com volumes reais permanece sem comprovação.

4.9 Escalabilidade, manutenção, evolução e viabilidade

Pergunta do enunciado	Estratégia
Como adicionar novos módulos?	Novo contexto com porta pública, dados próprios e consumidores de eventos existentes
Como integrar novos sistemas?	Adaptador para a porta existente e mapeamento de ID legado
Como alterar sem afetar tudo?	Mudança restrita ao contexto proprietário; contratos aditivos
Como lidar com mais usuários?	Réplicas da API sem estado atrás de balanceador
Como lidar com mais dados?	Índices, paginação, arquivamento e, com métricas, particionamento
Como reduzir o impacto de alterações?	Versionamento de contratos, testes de contrato e ADR
Como atualizar gradualmente?	Migrações aditivas (ADR-002), campos opcionais e nova versão para quebra

A extração de um contexto para serviço próprio só ocorre quando métricas de volume, disponibilidade ou cadência de implantação justificarem, sempre com novo ADR.

4.9.1 Visão de negócio e viabilidade

Público-alvo: áreas comercial, de contratos, financeira, de atendimento e de operações, além dos clientes que deixam de repetir informações. Proposta de valor: um cliente, um contrato e um chamado com a mesma identidade em todos os sistemas, com efeitos automáticos e rastreáveis. Benefícios esperados: menos retrabalho e duplicidade, cobrança no momento da ativação, SLA correto no atendimento e diagnóstico mais rápido.

Viabilidade: o monólito modular limita custo de infraestrutura e complexidade operacional, e a demonstração usa serviços sem licença ou em plano gratuito. Gratuidade não é custo zero: o esforço de implementação e de operação existe e não foi medido. A equipe júnior consegue manter um único implantável com fronteiras testadas. Custos, prazos e infraestrutura de produção não foram informados; vencimento em 10 dias, faturamento por ciclos iniciados, SLA contado da abertura e preparação de 72 h são premissas a validar; a adoção real depende de levantar a linha de base dos indicadores (tempo entre ativação e cobrança, recadastros, chamados com SLA incorreto e tempo de diagnóstico). Como afirma o enunciado, "uma boa arquitetura não é apenas tecnicamente adequada, mas também precisa fazer sentido para o negócio" (ARCHCORP, 2026a, p. 8).

4.10 Análise dos resultados

A versão analisada é 534eab2, que parte da integração das tarefas T06–T19 (PROJETO APLICADO 7, 2026b) e incorpora as correções de fechamento descritas nesta seção (PROJETO APLICADO 7, 2026d). Os números a seguir foram gerados automaticamente a partir das execuções, sem transcrição manual (PROJETO APLICADO 7, 2026a).

Verificação	Resultado
Suíte Python	269 aprovados e 14 ignorados (SQLite)
Interface (node:test)	19 testes aprovados
Contrato OpenAPI	sincronizado com a aplicação
Desempenho (RNF-07)	maior p95 de 21,53 ms em 10 consultas; meta de 500 ms
CI do GitHub Actions	a CI do pull request executa os jobs de aplicação, PostgreSQL 16, RabbitMQ e navegador

O relatório operacional transforma os dados já gravados pela API em informação para decisão: funil F1→F2→F3, cumprimento de SLA, atraso do despacho por consumidor, retentativas e falhas por evento, repetições absorvidas pela idempotência, p95 e erros por código e uso por módulo. No ensaio, o funil indicou que nenhuma fatura foi paga, porque os roteiros não exercitam pagamento; esse tipo de leitura orienta o que demonstrar e onde evoluir (PROJETO APLICADO 7, 2026c).

Nos cenários exercitados, os três fluxos evitam recadastro, automatizam cobrança e preparação de retirada e levam o SLA ao atendimento. Os resultados sustentam a viabilidade do desenho, mas não garantem convergência em toda ordem de eventos ou consistência de todos os estados. Os componentes estão organizados por propriedade de dados, com teste estático das fronteiras. REST atende respostas imediatas e eventos coordenam efeitos posteriores. Portas, contratos aditivos e ADRs permitem evolução com mudanças localizadas.

O fechamento corrige três falhas reproduzidas e protegidas por regressões (PROJETO APLICADO 7, 2026d). No F3, resolução ou reconciliação sem processo existente falham de forma recuperável e não confirmam a inbox; após recuperar a abertura, o despacho conclui o processo, com reprocessamento explícito quando o evento chegou a FAILED. No encerramento de contrato, tarefas ainda abertas são canceladas e o estado do processo deriva do agregado: todas DONE resultam em COMPLETED; qualquer tarefa CANCELLED entre tarefas finais resulta em CANCELLED. O cancelamento financeiro usa o mesmo estado efetivo da consulta: fatura vencida é recusada com 409, mesmo antes do registro explícito da inadimplência. As mudanças não reparam retroativamente registros consumidos em versões anteriores.

Riscos e limitações: os dados organizacionais reais são desconhecidos; não há OIDC, tracing distribuído nem agendador (vencimento de fatura e SLA dependem de acionamento); o RabbitMQ recebe cópia dos eventos, mas não

entrega aos consumidores, e FAILED é um estado da aplicação, não uma fila de erro do broker. O prazo da leitura do CRM limita tentativas e esperas, mas não interrompe uma chamada no mesmo processo; por isso o ADR-006 não foi adotado e fica como evolução para um CRM externo. A demonstração é local e a publicação pública ficou como evolução (ADR-007). Integração na main não comprova aceite docente, validação das premissas ou conclusão formal da entrega.

5 Considerações finais

Os objetivos técnicos possuem evidências no nível de protótipo acadêmico: requisitos, AS-IS, TO-BE, padrões, três integrações, interoperabilidade, qualidade e evolução estão documentados e demonstrados no recorte de validação apresentado. A autoria e as contribuições individuais estão identificadas na metodologia. A entrega acadêmica permanece condicionada à revisão docente das premissas e ao ensaio no ambiente escolhido. As decisões mais relevantes foram o monólito modular com fronteiras extraíveis, a outbox com inbox para efeitos únicos e o tratamento explícito da indisponibilidade (estado pendente e reconciliação no F3). Os principais desafios foram manter fronteiras com banco e processo compartilhados e garantir a mesma correlação em HTTP, auditoria e eventos.

As correções de convergência e coerência de estados da seção 4.10 são protegidas por testes de regressão. A prioridade passa a ser o ensaio completo e a validação do ambiente de apresentação. Como evolução, a arquitetura pode incorporar identidade corporativa, agendador para vencimentos e alertas de SLA, consumo efetivo pelo broker e conectores reais por adaptadores. Funcionalidades futuras incluem gestão de frota, vistoria com fotos e portal do cliente. O principal aprendizado é que idempotência, ordenação de eventos, estados de recuperação e evidências precisam ser projetados junto com cada fluxo.

6 Referências

ARCHCORP. Projeto Aplicado – ArchCorp: Arquitetura de Sistemas Corporativos. Enunciado acadêmico do Projeto Aplicado. 2026a.

ASYNCAPI INITIATIVE. AsyncAPI Specification 3.0.0. 2023. Disponível em: https://www.asyncapi.com/docs/reference/specification/v3.0.0. Acesso em: 17 out. 2026.

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 4. ed. Boston: Addison-Wesley, 2021.

BUTTLE, Francis; MAKLAN, Stan. Customer relationship management: concepts and technologies. 4. ed. London: Routledge, 2019.

COCKBURN, Alistair. Hexagonal architecture. 2005. Disponível em: https://alistair.cockburn.us/hexagonal-architecture/. Acesso em: 17 out. 2026.

DAVENPORT, Thomas H. Putting the enterprise into the enterprise system. Harvard Business Review, Boston, v. 76, n. 4, p. 121-131, jul./ago. 1998.

DUMAS, Marlon; LA ROSA, Marcello; MENDLING, Jan; REIJERS, Hajo A. Fundamentals of business process management. 2. ed. Berlin: Springer, 2018.

FIELDING, Roy; NOTTINGHAM, Mark; RESCHKE, Julian (ed.). RFC 9110: HTTP semantics. Internet Engineering Task Force, jun. 2022. Disponível em: https://www.rfc-editor.org/rfc/rfc9110. Acesso em: 17 out. 2026.

FOWLER, Martin. Patterns of enterprise application architecture. Boston: Addison-Wesley, 2002.

FOWLER, Martin. MonolithFirst. 3 jun. 2015. Disponível em: https://martinfowler.com/bliki/MonolithFirst.html. Acesso em: 17 out. 2026.

HOHPE, Gregor; WOOLF, Bobby. Enterprise integration patterns: designing, building, and deploying messaging solutions. Boston: Addison-Wesley, 2003.

ISO/IEC. ISO/IEC 25010:2011: systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models. Geneva: ISO, 2011.

LEHMAN, Meir M. Programs, life cycles, and laws of software evolution. Proceedings of the IEEE, v. 68, n. 9, p. 1060-1076, 1980.

LEWIS, James; FOWLER, Martin. Microservices: a definition of this new architectural term. 25 mar. 2014. Disponível em: https://martinfowler.com/articles/microservices.html. Acesso em: 17 out. 2026.

NEWMAN, Sam. Building microservices: designing fine-grained systems. 2. ed. Sebastopol: O'Reilly Media, 2021.

NYGARD, Michael T. Release it!: design and deploy production-ready software. 2. ed. Raleigh: Pragmatic Bookshelf, 2018.

OPENAPI INITIATIVE. OpenAPI Specification 3.1.1. 2024. Disponível em: https://spec.openapis.org/oas/v3.1.1.html. Acesso em: 17 out. 2026.

PROJETO APLICADO 7. Evidências de validação. Documento docs/EVIDENCIAS_VALIDACAO.md e arquivo output/evidencias/evidencias.json, versão 534eab2, 29/10/2026. 2026a.

PROJETO APLICADO 7. Integra as tarefas T06–T19 à main e recupera correções do T17. PR #75, commit f1669d78f01e2d279b3de60a999ed778e6c9916e, 25 out. 2026. Disponível em: https://github.com/Roger-Quinelato/projetoAplicado7/pull/75. Registro do commit consultado localmente em: 28 out. 2026. 2026b.

PROJETO APLICADO 7. Relatório operacional e agentes de verificação. Arquivos scripts/relatorio_operacional.py, tools/consolidar_evidencias.py e docs/agentes/AGENTES_FDE_QA.md. 2026c.

PROJETO APLICADO 7. Corrige recuperação do F3, estados de fechamento e autoria da entrega. PR #78, commit 714b17c, 28 out. 2026. Disponível em: https://github.com/Roger-Quinelato/projetoAplicado7/pull/78. 2026d.

RICHARDS, Mark; FORD, Neal. Fundamentals of software architecture: an engineering approach. Sebastopol: O'Reilly Media, 2020.

Documentos complementares do repositório: ADR-001 a ADR-007; requisitos; plano de implementação; matriz de rastreabilidade; arquitetura AS-IS, TO-BE e fluxos de integração; exemplos de API; contratos OpenAPI e AsyncAPI; roteiros de demonstração; evidências de validação.